OS Unit V - File Management

Unit V – File Management

Operating System | MSBTE K-Scheme | Diploma CO Branch | TY

5.1 File Concepts

A **file** is a named collection of related information (logically related data or program instructions) recorded on secondary storage, such as a hard disk. Files are the most visible unit of logical secondary storage provided by the OS — data cannot be written to secondary storage unless it is within a file.

The OS abstracts the physical properties of storage devices to define a **logical storage unit**, the file, which is managed by the **File System**.

File Attributes

Every file has certain attributes (properties), which vary somewhat across operating systems but generally include:

1. **Name** – the symbolic name of the file (the only information kept in human-readable form).
2. **Identifier** – a unique tag (usually a number) that identifies the file within the file system; it is the non-human-readable name for the file.
3. **Type** – needed for systems that support different file types (e.g., .txt, .exe, .docx).
4. **Location** – a pointer to the device and to the location of the file on that device.
5. **Size** – the current size of the file (in bytes/words/blocks), and possibly the maximum allowed size.
6. **Protection** – access-control information that determines who can read, write, or execute the file.
7. **Time, Date, and User Identification** – data kept for creation, last modification, and last use; useful for protection, security, and usage monitoring.

This information about all files is stored in the **directory structure**, which also resides on secondary storage.

File Operations

The OS provides system calls to perform basic operations on files:

1. **Creating a file** – Space for the file is found in the file system, and an entry for the new file is made in the directory.
2. **Writing a file** – Using a system call along with the file name and the information to be written; the system keeps a **write pointer** to the location where the next write is to take place.
3. **Reading a file** – Uses a system call specifying the file name and where the next block of the file should be put; the system maintains a **read pointer**.
4. **Repositioning within a file (seek)** – The directory is searched for the entry, and the current-file-position pointer is repositioned to a given value, without doing any actual I/O.
5. **Deleting a file** – The directory entry for the file is searched; once found, all associated file space is released, and the directory entry is erased.
6. **Truncating a file** – Erases the contents of a file but keeps its attributes, resetting its length to zero, without actually deleting the file itself.

Other common operations: appending, renaming, copying.

File Types

Operating systems support different types of files, typically identified through the **file name and extension**:

File Type	Usual Extension	Function
Executable	exe, com, bin	Ready-to-run machine-language program
Object	obj, o	Compiled, machine language, not linked
Source Code	c, java, py, asm	Source code in various languages
Batch	bat, sh	Commands to the command interpreter
Text	txt, doc	Textual data, documents
Word Processor	doc, wp, tex	Various word-processor formats
Archive	arc, zip, tar	Related/grouped files, compressed
Multimedia	mpeg, mov, mp3, mp4	Binary file with audio/video info

Note: The extension helps the OS (and applications) recognize the type of the file and select an appropriate application to open it.

File System Structure

A **file system** provides efficient and convenient access to the disk by allowing data to be stored, located, and retrieved easily. It is typically organized in layers:

1. **Application Programs** – (topmost layer) issue requests for file operations.
2. **Logical File System** – manages metadata (all file-system structure except the actual data/content), such as the directory structure, and provides protection.
3. **File-Organization Module** – knows about files and their logical blocks, as well as physical blocks; translates logical block addresses to physical block addresses.
4. **Basic File System (Basic I/O Supervisor)** – issues generic commands to the device driver to read/write physical blocks on the disk.
5. **I/O Control (Device Drivers)** – the lowest level; consists of device drivers and interrupt handlers to transfer data between main memory and the disk system.
6. **Devices** – the physical hardware (disk).

Diagram (described):

```
Application Programs
        |
Logical File System
        |
File-Organization Module
        |
Basic File System
        |
I/O Control (Device Drivers)
        |
   Devices (Disk)
```

Each layer uses the features of the lower layer to create new features for use by higher layers, giving the file system a clean, modular structure.

5.2 Accessing Methods

Files store information that must be accessed and read into computer memory. There are different ways in which information in a file can be accessed:

Sequential Access

- The **simplest** access method — information is processed **in order**, one record after another.
- A **read** operation reads the next portion of the file and automatically advances a file pointer; a **write** operation appends to the end of the file and advances the pointer.
- Modeled on tape-based storage systems, but is common with disk files as well.

Operations supported: `read next`, `write next`, `reset` (to the beginning), and sometimes `skip n records`.

Use case: Suitable for applications like editors and compilers that process files from beginning to end (e.g., reading a text file line by line).

Direct Access (Random Access)

- Also called **Relative Access**.
- A file is made up of **fixed-length logical records** that allow programs to read and write records **rapidly, in any order**.
- The direct-access method is based on a **disk model** of a file, since disks allow random access to any file block.
- The file is viewed as a numbered sequence of blocks or records; a program can jump directly to record number `n` without reading records `0` through `n-1` first.

Operations supported: `read n` (read the record numbered n), `write n` (write the record numbered n), where `n` is a relative block/record number.

Use case: Essential for applications requiring immediate access to large amounts of information, such as **databases**, where a specific record needs to be retrieved without scanning the entire file.

Comparison Table: | Sequential Access | Direct Access | |—|—| | Records accessed in order | Records accessed in any order | | Based on tape model | Based on disk model | | Simple, slower for random lookups | Faster for random lookups | | Used in editors, compilers | Used in databases, indexed files |

Note: Other access methods (like indexed access) can be built on top of a direct-access file by constructing an index.

5.3 File Allocation Methods

The allocation method defines **how disk blocks are allocated to files**, so that disk space is used effectively and files can be accessed quickly. Three major methods:

1. Contiguous Allocation

- Each file occupies a set of **contiguous (consecutive) blocks** on the disk.
- The directory entry for a file stores the **address of the starting block** and the **length (number of blocks)** of the file.

Advantages: - Simple to implement — only the starting location and length are needed. - **Supports both sequential and direct access efficiently**, since the block number of any record can be calculated directly from the starting address. - Fast, as it requires minimal head movement/seeking for reading a file in order.

Disadvantages: - Suffers from **external fragmentation**, as files are deleted and created, leaving scattered free holes. - Difficult to find contiguous space for a **growing file** — if a file needs to expand and the adjacent block is already allocated to another file, the whole file may need to be relocated to a larger space. - Requires the OS to know the **maximum file size in advance** for pre-allocation strategies, which is often not possible.

---

2. Linked Allocation

- Each file is a **linked list of disk blocks**, which may be scattered anywhere on the disk.
- The directory contains a pointer to the **first** and (sometimes) **last** block of the file.
- Each block contains a pointer to the **next block** in the file; the final block contains a **null pointer**.

Advantages: - Solves external fragmentation completely, since any free block can be used. - No need to declare the file size in advance; a file can grow as long as free blocks are available.

Disadvantages: - **Only effective for sequential access**; direct access is inefficient since to access block `i`, the system must start at the beginning of the file and follow the pointers `i` times. - Space is wasted for storing pointers in each block. - **Reliability issue**: if a pointer is lost or damaged (e.g., due to a hardware failure), the rest of the file (or the whole disk, if the corruption creates loops) can become inaccessible.

Variation – File Allocation Table (FAT): - A section of disk at the beginning of each partition (the FAT) is set aside to contain the linked-list pointers instead of storing them inside each block. - Each entry is indexed by block number and contains the number of the **next block** in the file (or an end-of-file marker). - This improves performance since the FAT can be cached in memory, avoiding repeated disk head movement for pointer lookups.

---

3. Indexed Allocation

- Solves the problems of linked allocation (no direct access) by bringing all the block pointers together into one location: an **index block**.
- Each file has its own **index block**, an array of disk-block addresses.
- The `i`-th entry in the index block points to the `i`-th block of the file.
- The directory contains the address of the index block for the file.

Advantages: - Supports **direct access** efficiently, since the address of any block can be found directly from the index block, without following a chain of pointers. - No external fragmentation.

Disadvantages: - **Wasted space**: for very small files, having an entire index block (which may be much larger than needed) is wasteful overhead compared to linked allocation. - **Limits on file size**: since an index block can only hold a limited number of pointers, very large files require a scheme to combine multiple index blocks, such as: - **Linked scheme** – linking several index blocks together. - **Multilevel index** – an index block points to other (secondary) index blocks, which point to the actual data blocks. - **Combined scheme** – using a mix of direct pointers (for small files) and indirect pointers (single, double, triple indirect blocks for larger files) — this approach is used in traditional UNIX/Linux inode structures.

Comparison Table: | Allocation Method | Sequential Access | Direct Access | External Fragmentation | Overhead | |—|—|—|—|—| | Contiguous | Fast | Fast | Yes | Low (start + length) | | Linked | Fast | Slow (must follow pointers) | No | Pointer per block | | Indexed | Fast | Fast | No | Index block per file |

5.4 Directory Structure

A **directory** is a structure that organizes and provides information about all the files in the system, such as their name, location, size, and type. The directory itself can be thought of as a symbol table that translates file names into their directory entries.

1. Single-Level Directory

- The **simplest** directory structure — all files are contained in the **same single directory**, which is easy to support and understand.

Limitations: - **Naming problem**: since all files are in one directory, they must all have **unique names** — this becomes difficult to manage when there are many files or many users sharing the same system. - **Grouping problem**: there is no way to group related files together (e.g., no separate folders for different users or projects).

Diagram (described):

```
Directory: file1, file2, file3, file4, ...  (all files listed flat, in one directory)
```

2. Two-Level Directory

- To resolve the naming problem, a **separate directory is created for each user**, called a **User File Directory (UFD)**.
- A **Master File Directory (MFD)** is indexed by user name, and each entry in the MFD points to that user's UFD.

Advantages: - Solves the **naming problem** — different users can have files with the same name, since each user has a private UFD. - Provides some **efficiency in searching**, since only the relevant user's UFD needs to be searched.

Limitations: - Still **isolates users from one another completely** — sharing files between users is difficult or requires special provisions. - Does not allow users to **group their own files** into further sub-categories (no sub-directories within a UFD).

Diagram (described):

```
Master File Directory (MFD)
    ├── User1 --> UFD1: file1, file2
    ├── User2 --> UFD2: file1, file3
    └── User3 --> UFD3: file2, file4
```

3. Tree-Structured Directory

- Generalizes the two-level directory into a **tree of arbitrary height**, allowing users to create their own **subdirectories** to group and organize related files.
- Each user (and the system) has a **root directory**; every file has a **unique path name** from the root directory to that file.

Features: - Directories (or folders) themselves are treated as special files/entries, and can contain other directories and files. - Every process/user has a concept of a **"current directory"** (working directory), so files can be referred to by **relative path names** (relative to the current directory) or **absolute path names** (starting from the root). - Allows efficient **searching**, logical **grouping** of related files, and better organizational flexibility.

Operations supported: - **Creating** a new directory/subdirectory. - **Deleting** a directory — usually only allowed if the directory is empty (some systems allow recursive deletion of a non-empty directory, deleting all its contents as well).

Diagram (described):

```
Root
 ├── User1
 │    ├── Projects
 │    │    ├── file1
 │    │    └── file2
 │    └── Documents
 │         └── file3
 └── User2
      └── file4
```

Advantages over two-level: - Solves the grouping problem, since users can organize files into subdirectories as needed. - More scalable and closely matches modern real-world file systems (e.g., Windows, Linux).

Comparison Table: | Single-Level | Two-Level | Tree-Structured | |—|—|—| | One directory for all files | One directory per user (UFD) under a Master directory (MFD) | Hierarchical, unlimited nested subdirectories | | Naming conflicts likely | Naming conflicts avoided across users | Naming conflicts avoided; flexible grouping | | No grouping possible | Limited grouping (per user only) | Full grouping via subdirectories | | Very simple,

limited use | Simple, still limited for large systems | Used in most modern operating systems |

Quick Revision Summary

- A **file** is a named collection of related data; **attributes** include name, identifier, type, location, size, protection, and timestamps.
- **File operations**: create, read, write, seek (reposition), delete, truncate.
- **File types**: executable, object, source code, text, archive, multimedia, etc. (identified by extension).
- **File system structure**: layered — Application Programs → Logical File System → File-Organization Module → Basic File System → I/O Control → Devices.
- **Access methods**: Sequential (in order, tape-based model) and Direct (random access, disk-based model, used in databases).
- **Allocation methods**: Contiguous (fast, but external fragmentation), Linked (no fragmentation, but poor direct access; improved via FAT), Indexed (supports direct access via index blocks; may need multilevel/combined schemes for large files).
- **Directory structures**: Single-Level (simple, naming conflicts), Two-Level (per-user UFD under MFD, no grouping), Tree-Structured (hierarchical, subdirectories, path names — used in modern OS).